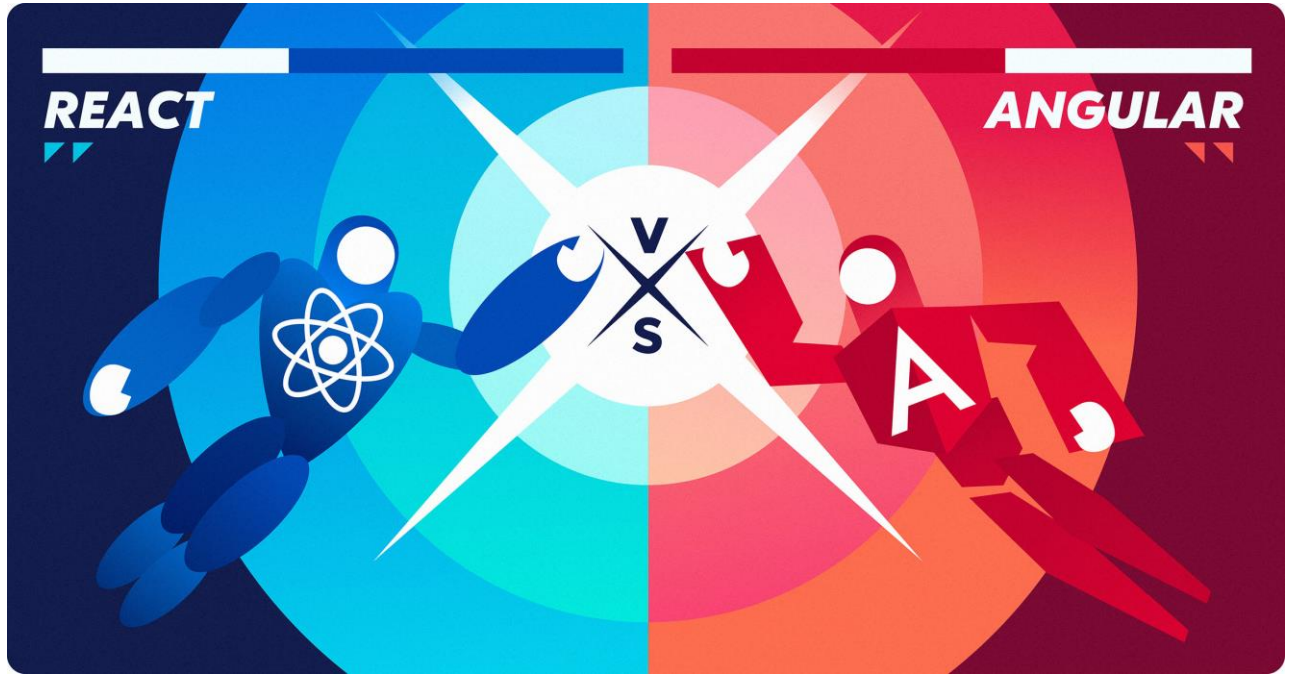


TECHNOLOGIES WEB II (React & Angular)

Professeur	Dr. Fadwa LACHHAB
Niveau	4eme année
Année	2024-2025

Partie 2 : Angular

Semaine 1
- Presentation de Angular
- Environnement de développement
- Components

Semaine 2
- Data Binding/Directives
- Observables/ Forms

Semaine 3
- Services / Routing

Semaine 4
- Interaction avec Base de Données : FireBase

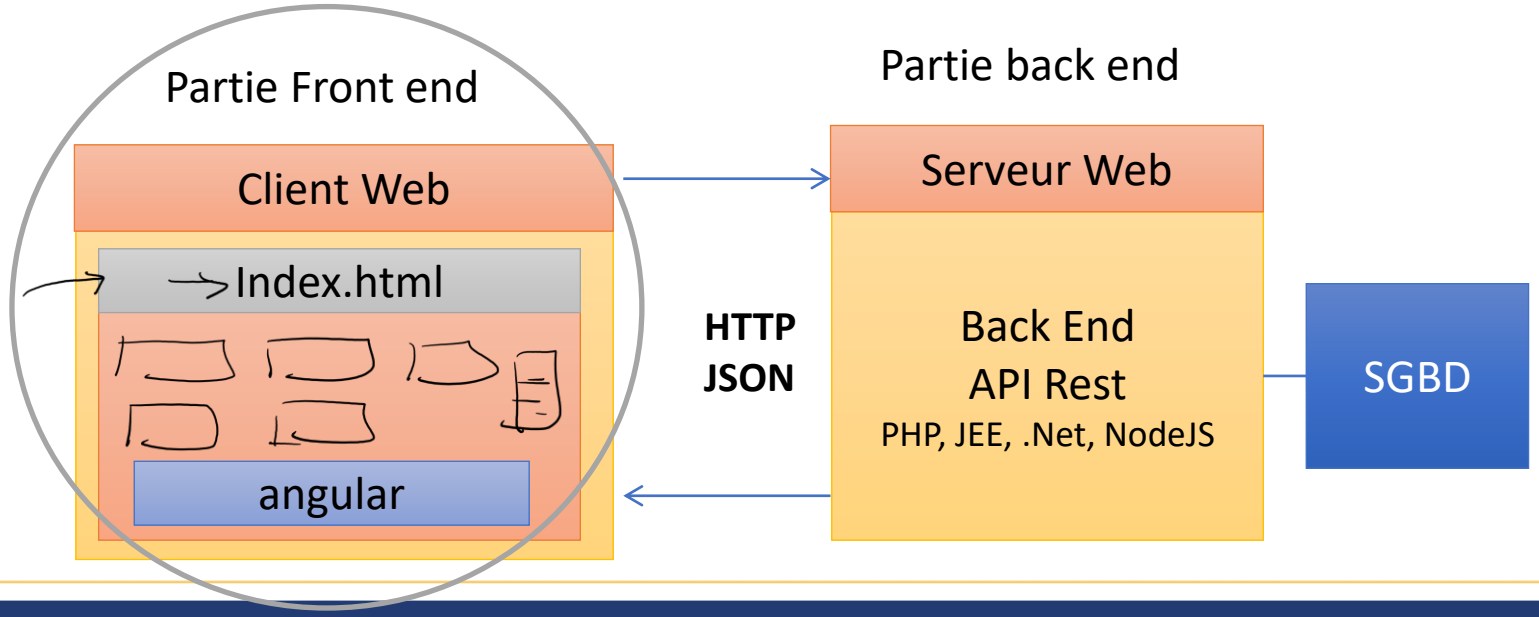
Semaine 5
- Pipes /HTTP
- Authentification

Semaine 6
- Examen TP

Angular

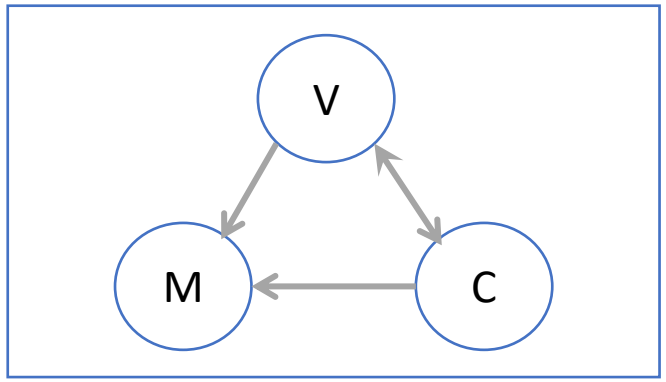
Angular

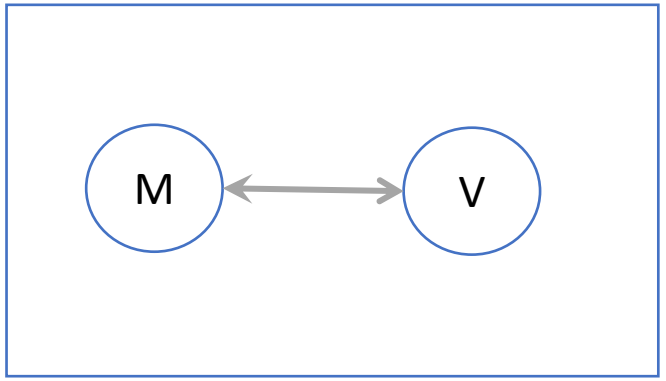
- **Angular** Permet de créer la partie front end des applications web de type **SPA** (Single Page Application réactive)
- Pour naviguer entre les différentes parties de cette application, **Java Script** est utilisé pour envoyer des requêtes http vers le serveur pour récupérer du contenu dynamique généralement au format JSON.
- Ce contenu **JSON** est ensuite affiché coté client au format **HTML** dans la même page. (Rendu HTML coté Client)



AngularJS / Angular

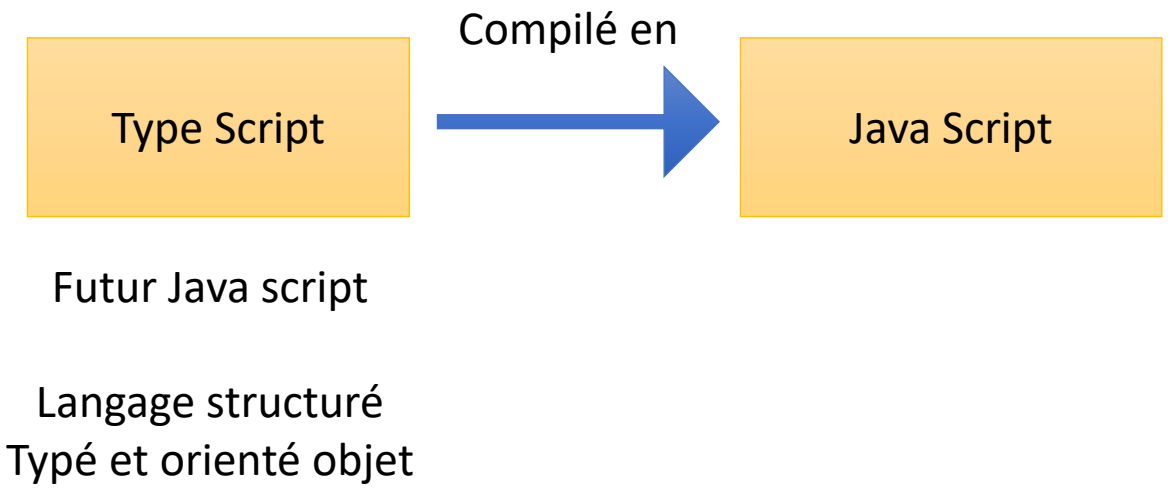
- **Angular 1** (Agular JS) :
  - Première version de Angular qui est la plus populaire.
  - Elle est basée sur une architecture MVC coté client. Les applications Angular 1 sont écrite en Java Script.
- **Angular 2** (Angular) :
  - Est une réécriture de Angular 1 qui est plus performante, mieux structurée et représente le futur de Angular.
  - Les applications de Angular2 sont écrites en Type Script qui est compilé et traduit en Java Script avant d'être exécuté par les Browsers Web.
  - Angular 2 est basée sur une programmation basée sur les Composants Web (Web Component)
- **Angular 4, 5, 6, …, 19**





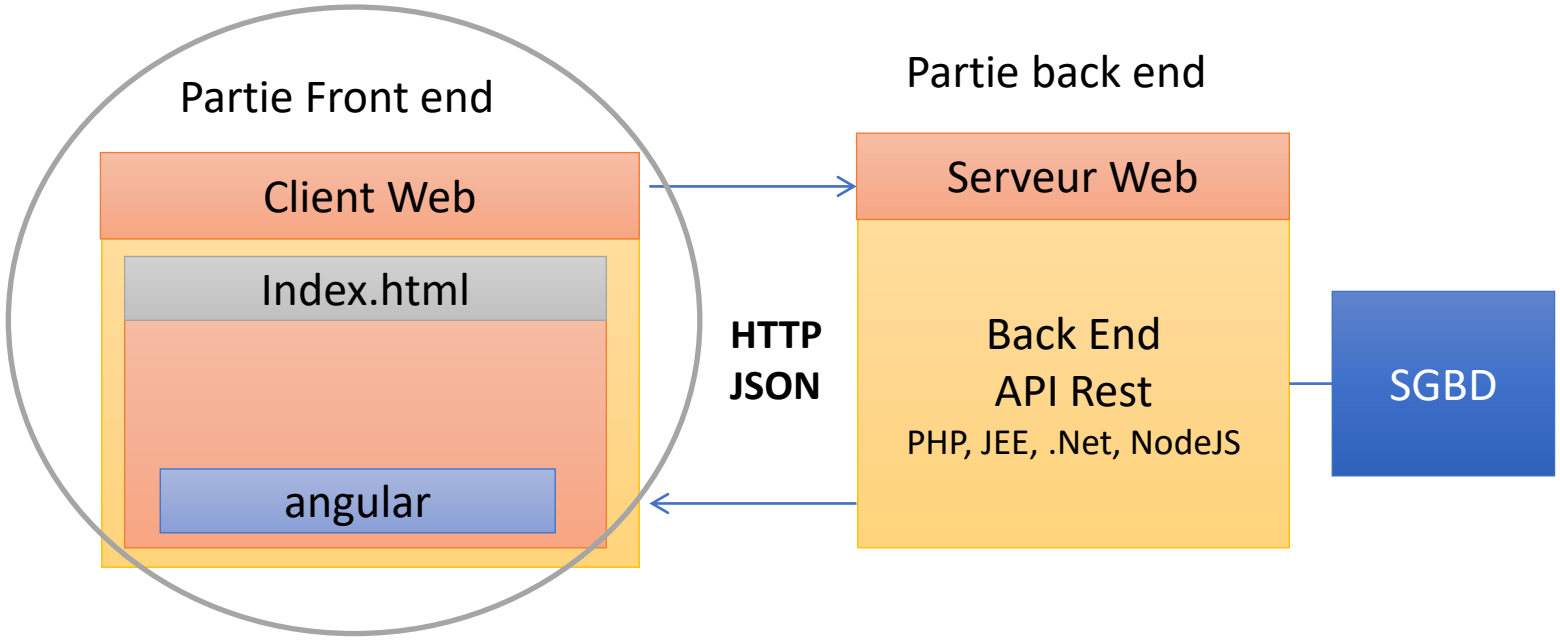
Angular avec Type Script

- Pour développer une application Angular il est recommandé d'utiliser Type Script qui sera compilé et traduit en Java Script.
- Type Script est un langage de script structuré et orienté objet qui permet de simplifier le développement d'applications Java Script



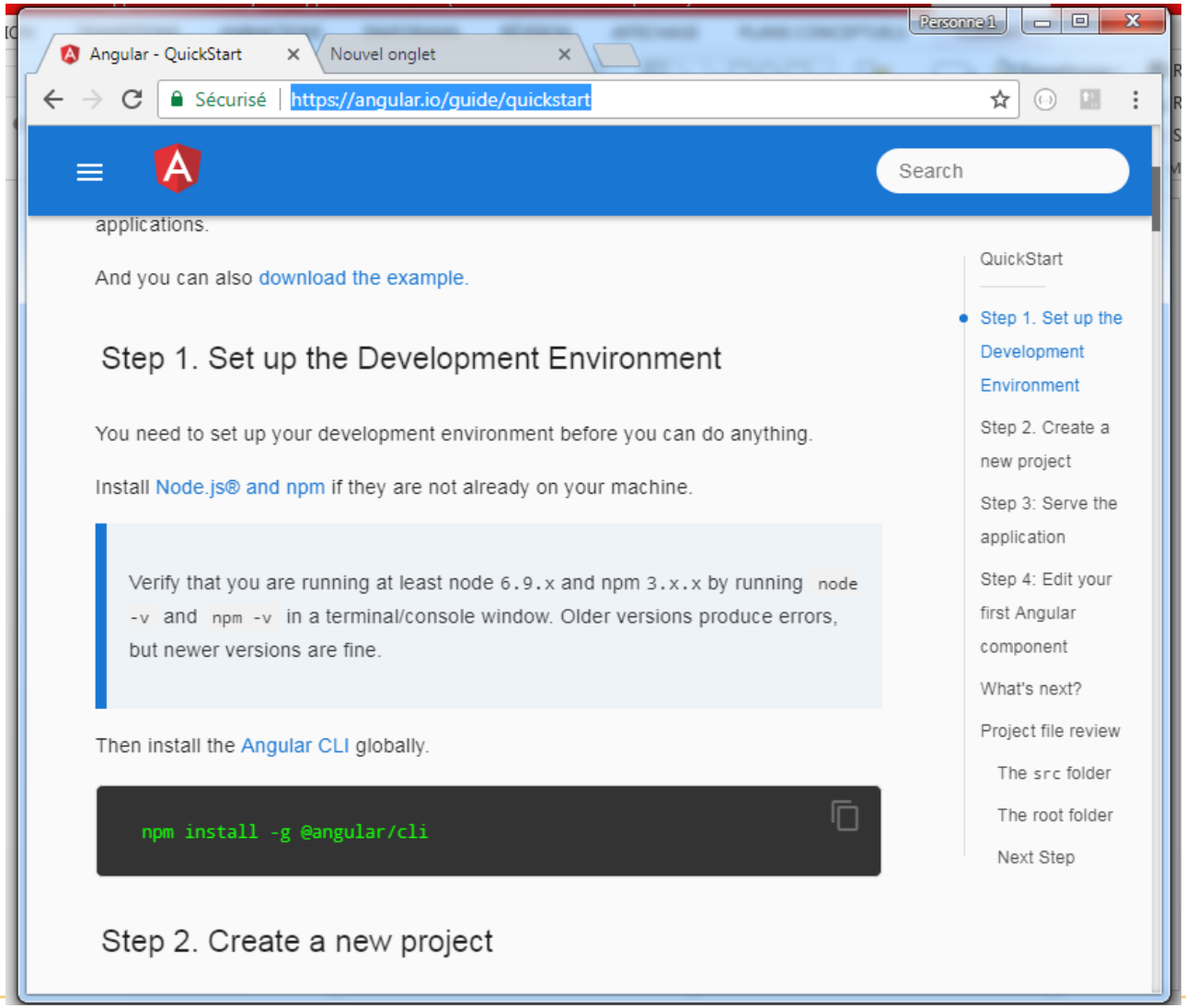
Single Page Application: SPA

- Angular Permet de créer la partie front end des applications web de type SPA (Single Page Application)



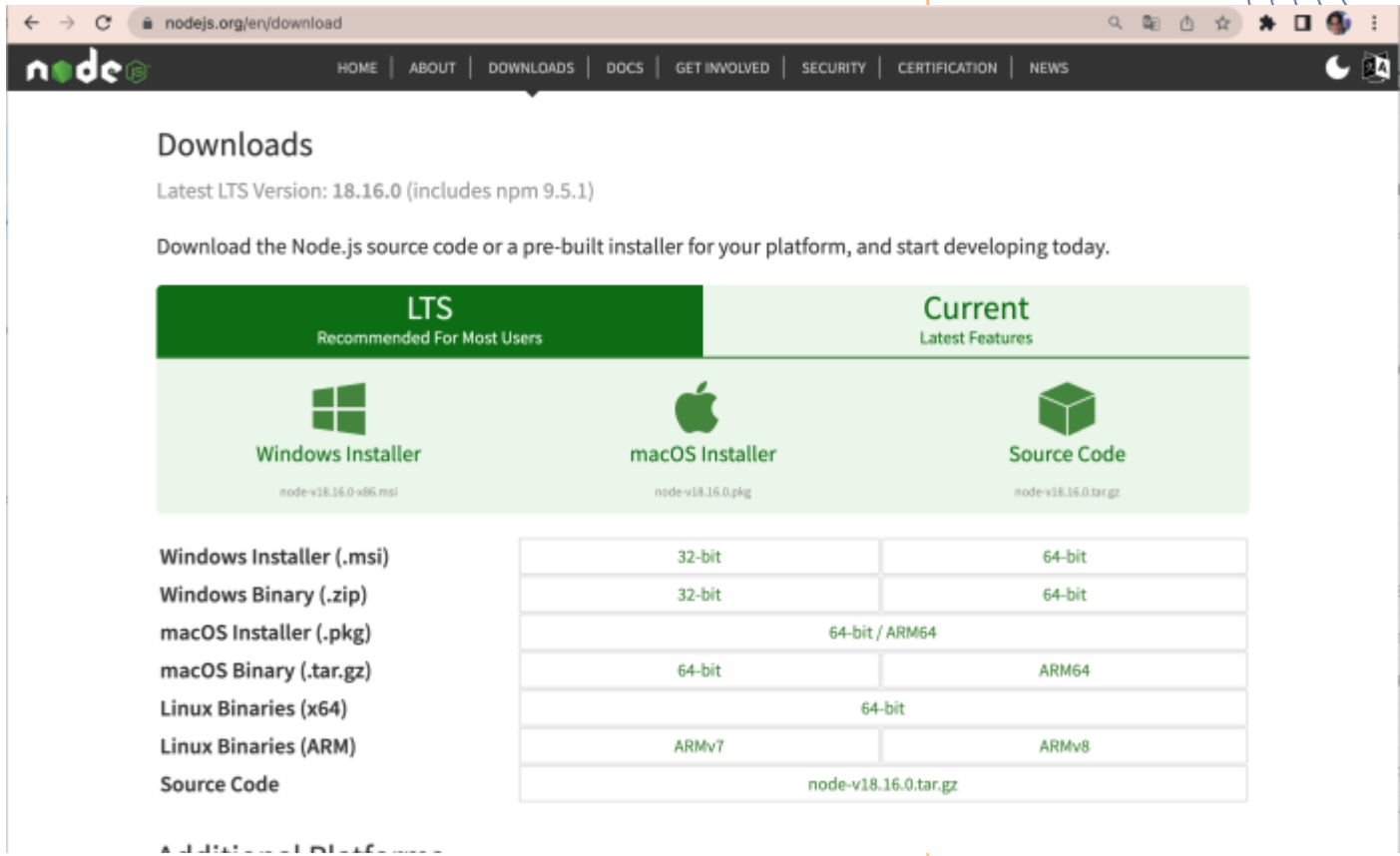
Démarre avec Angular

- https://angular.io/guide/quickstart



Installation des outils

- Pour faciliter le développement d'une application Angular, les outils suivants doivent être installés :
  - Node JS : https://nodejs.org/en/download/
    - Node JS installe l'outil npm (Node Package Manager) qui permet de télécharger et installer des bibliothèques Java Script.
  - Installer ensuite Angular CLI (Command Line Interface) qui vous permet de générer, compiler, tester et déployer des projets angular (https://cli.angular.io/) :
    - **npm install -g @angular/cli**

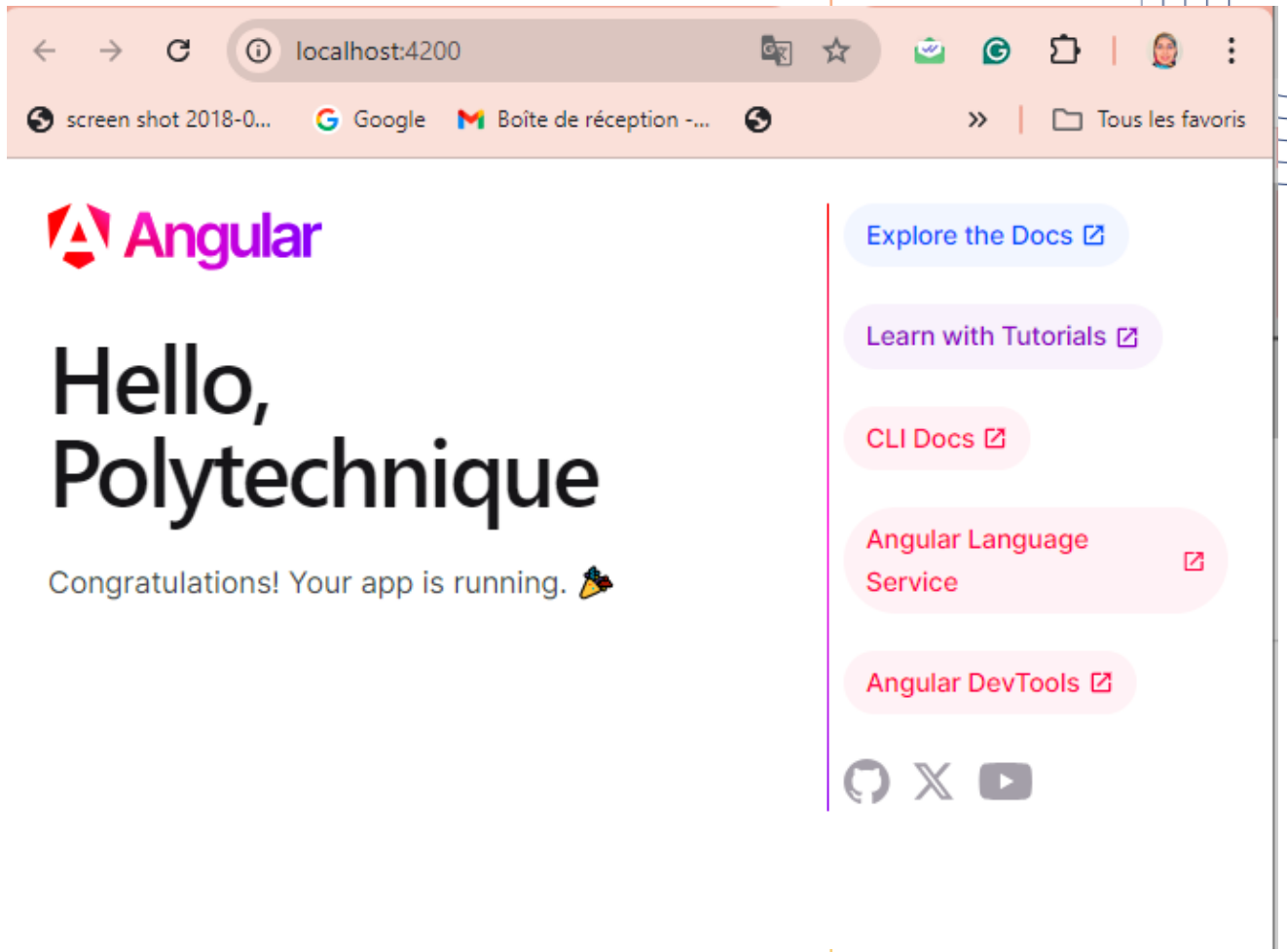


Création d'un nouveau projet Angular

- Pour générer la structure d'un projet Angular, on utilise Angular CLI via sa commande ng suivie des options new et le nom du projet.
  - > ng new FirstApp
- Cette commande génère les différents fichiers requis par une application basique Angular et installe aussi toutes les dépendances requises par ce projet.

Exécuter un projet Angular

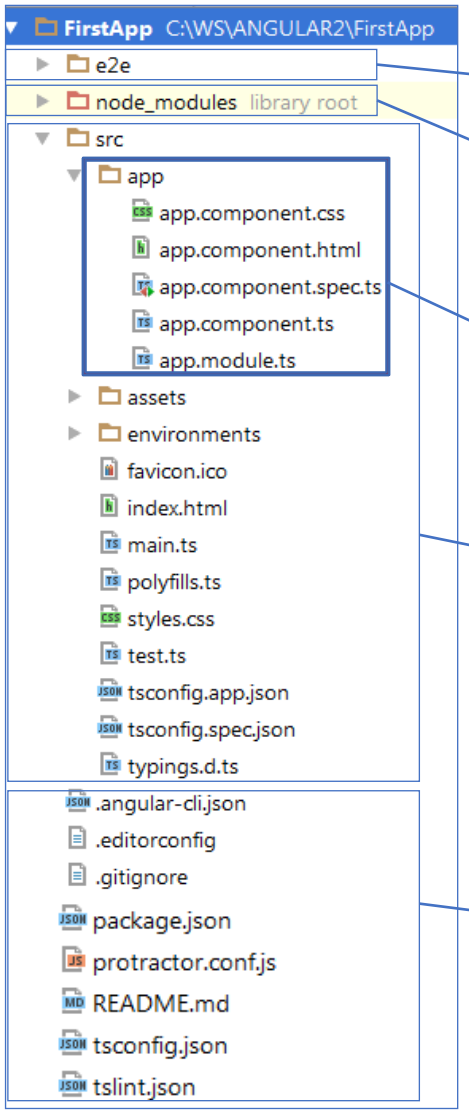
- Pour exécuter un projet **Angular**, on exécuter la commande suivante à partir de la racine du projet
  - `> ng serve`
- Cette commande compile le code source du projet pour transcompilé le code **TypeScript** en **Java Script** et en même temps démarre un serveur Web local basé sur NodeJS pour déployer l'application localement.
- Pour tester le projet généré, il suffit de lancer le Browser et taper l'url : **http:// localhost:4200**
- Dans l'étape suivante, nous allons regarder la structure du projet généré par Angular CLI.



Edition du projet

- Plusieurs IDE professionnels peuvent être utilisé pour éditer le code:
  - Intellij, Web Storm, PHP Storm
  - **Visual Studio Code**
- D’autres éditeurs classiques peuvent être aussi utilisé :
  - Atom
  - Sublime Text
  - Etc …

Structure du projet Angular



Fichier de configuration du projet

Dépendances externes du projets (Librairies Java Script et CSS)

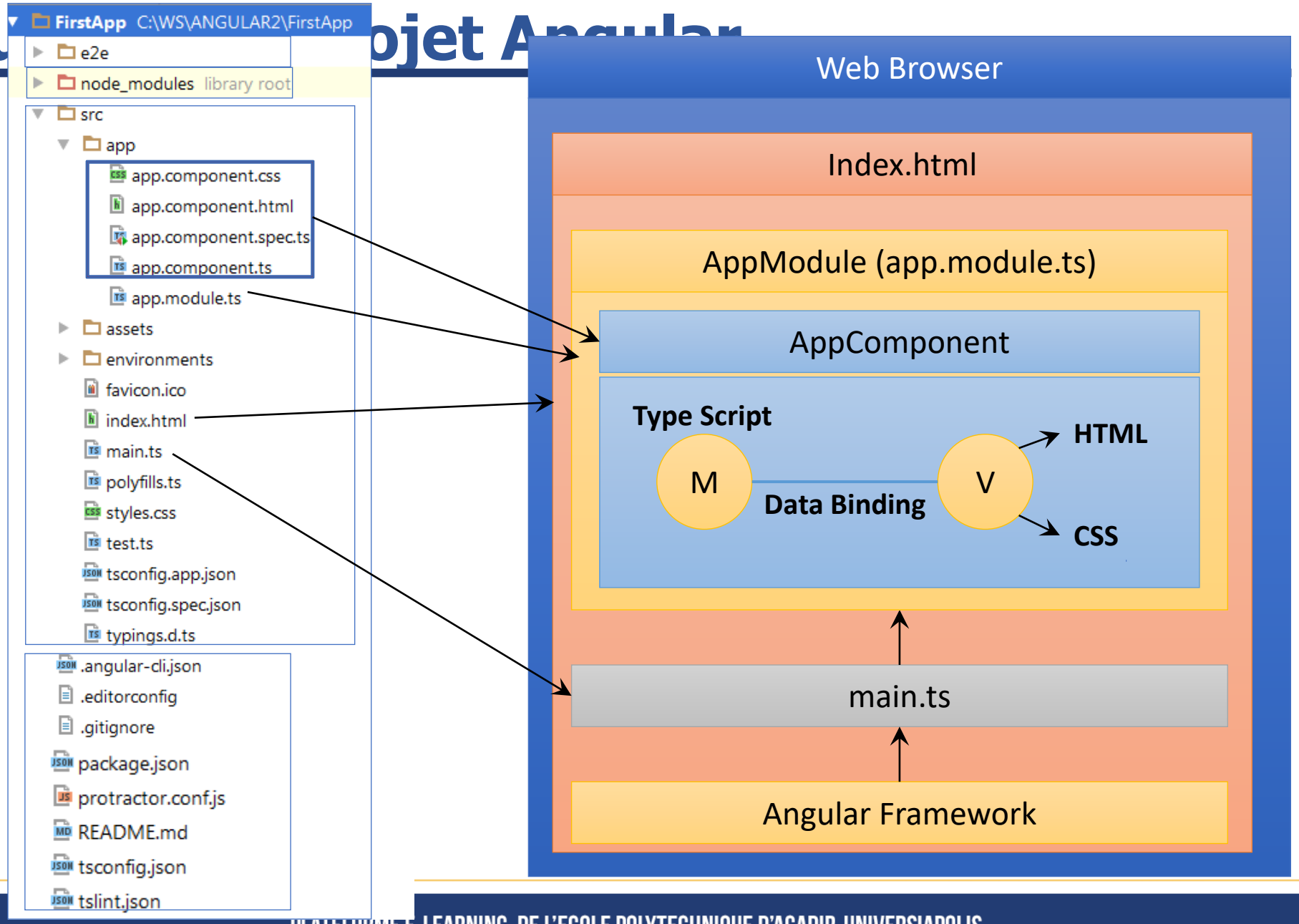
Logique applicative de votre projet : Les composants, Les services ..

C'est dans ce dossier app ou vous allez passer votre temps de dev

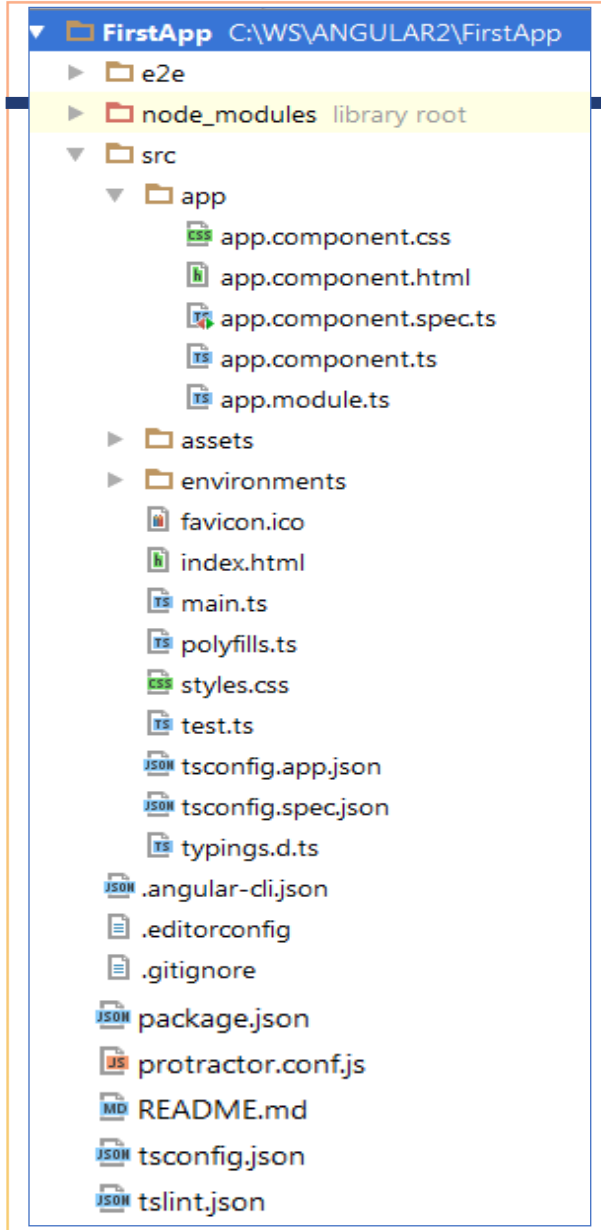
Code source relatif à votre projet

Fichier de configuration du projet

Stru[illegible]ojet A[illegible]


FirstApp C:\WS\ANGULAR2\FirstApp
e2e
node_modules library root
src
app
app.component.css
app.component.html
app.component.spec.ts
app.component.ts
app.module.ts
assets
environments
favicon.ico
index.html
main.ts
polyfills.ts
styles.css
test.ts
tsconfig.app.json
tsconfig.spec.json
typings.d.ts
.angular-cli.json
.editorconfig
.gitignore
package.json
protractor.conf.js
README.md
tsconfig.json
tslint.json
Web Browser
Index.html
AppModule (app.module.ts)
AppComponent
Type Script
M
Data Binding
V
HTML
CSS
main.ts
Angular Framework

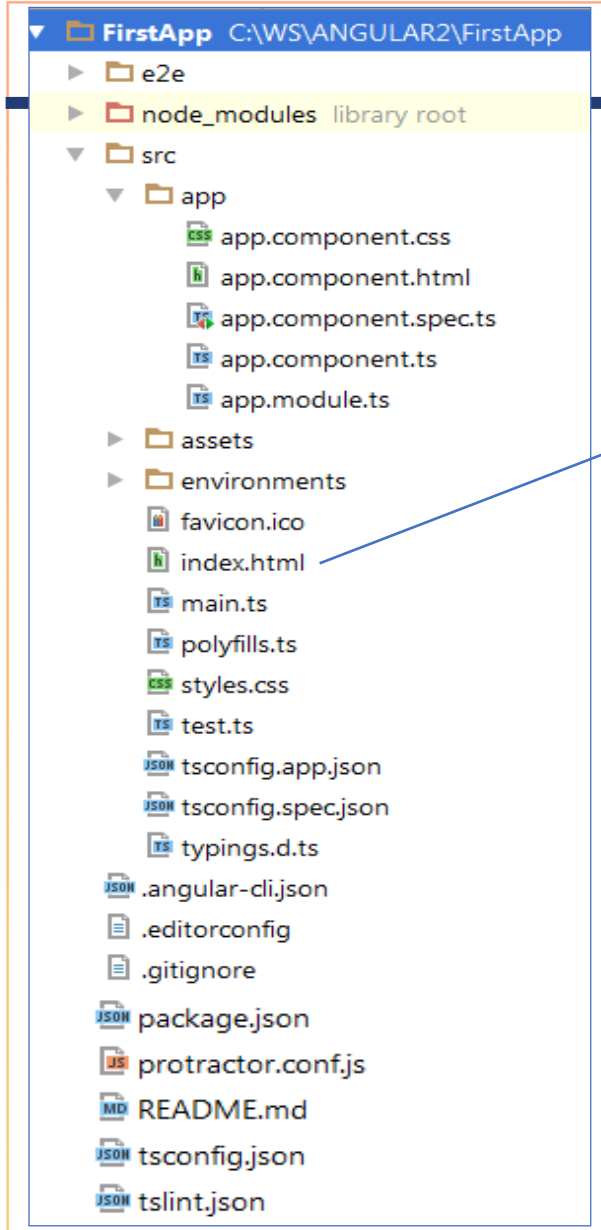
Structure du projet Angular



Dans Angular, nous avons plusieurs fichiers de configuration à savoir:

- **Package.json**: C'est un fichier de configuration NPM. Il contient des configurations Angular. Ces configurations permettent d'installer les autres packages en relation, en utilisant NPM.
- **tsconfig.json**: Tous les fichiers TypeScript (*.ts) seront compilés et convertis vers des fichiers JavaScript(*.js). Ce que permet, aux différents navigateurs, de lire ces fichiers. Pour assurer cette action on fait appel au fichier tsconfig.json.

Structure du projet Angular

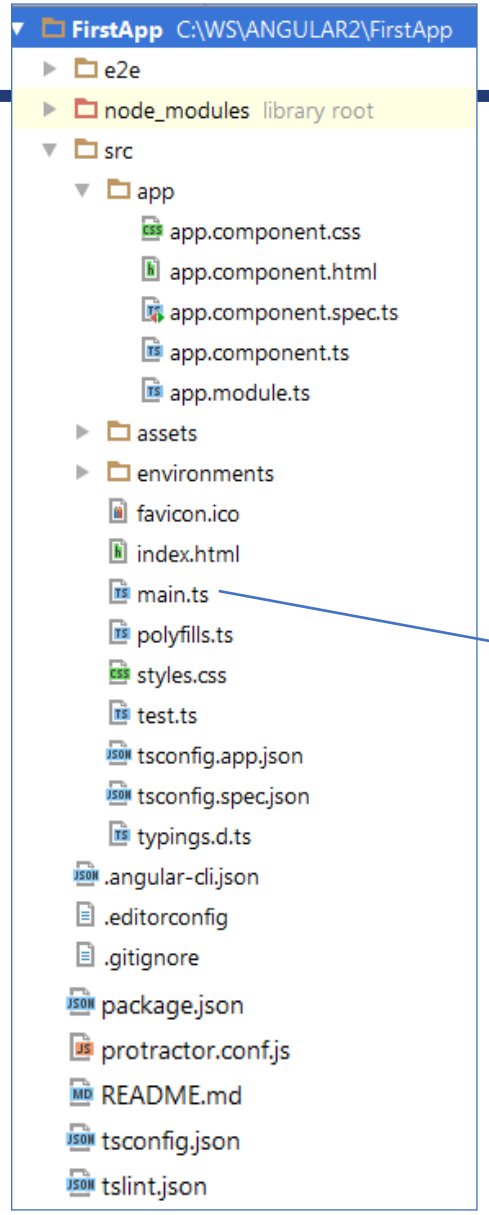


Index.html: c'est notre page de démarrage. Elle contient les appels vers les components Angular.

index.html

```
<!doctype html>
<html lang="en">
<head>
  <meta charset="utf-8">
  <title>FirstApp</title>
  <base href="/">

  <meta name="viewport" content="width=device-
width, initial-scale=1">
  <link rel="icon" type="image/x-icon"
href="favicon.ico">
</head>
<body>

  <app-root> </app-root>

</body>
</html>
```

Structure du projet Angular

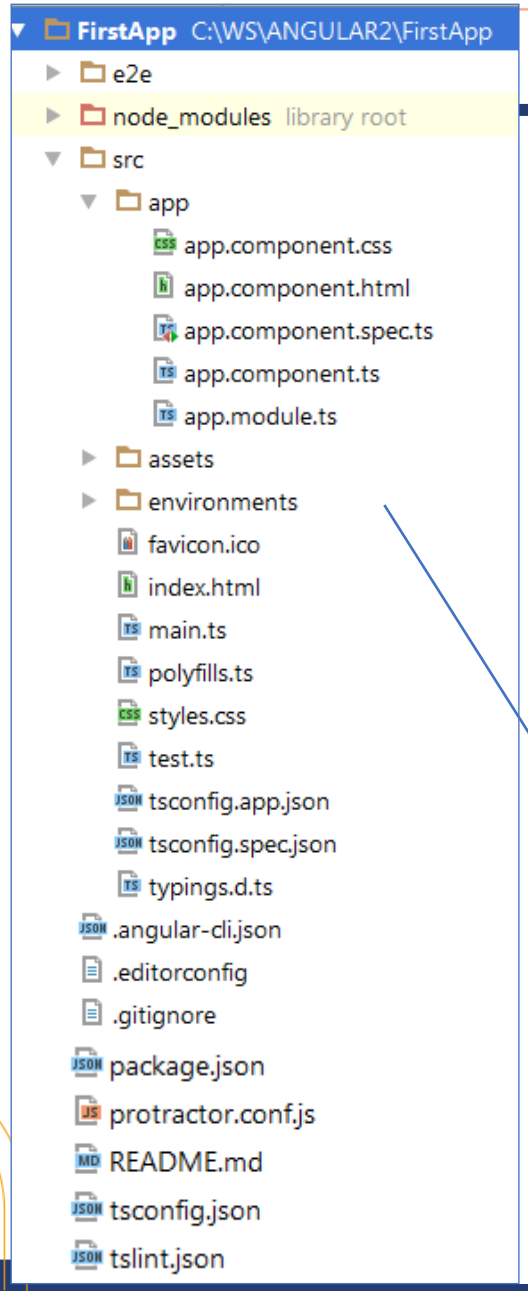


main.ts : est le fichier de démarrage. Il permet de définir les modules avec l'application qui va démarrer.

main.ts

```
import { enableProdMode } from '@angular/core';
import { platformBrowserDynamic } from
'@angular/platform-browser-dynamic';

import { AppModule } from './app/app.module';
import { environment } from
'./environments/environment';

if (environment.production) {
  enableProdMode();
}

platformBrowserDynamic().bootstrapModule(AppModule);
```

Structure du projet Angular

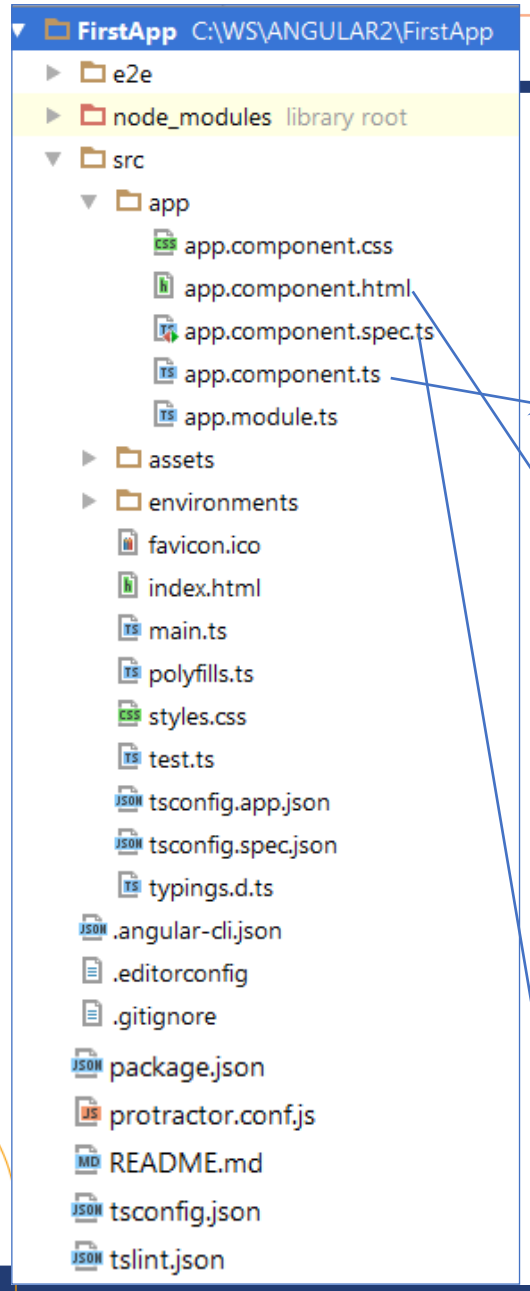


app.module.ts: Angular est modulaire. Chaque module est composé par un ensemble des components. Dans chaque module on déclare les components qui sont exposables.

app.module.ts

```
import { BrowserModule } from '@angular/platform-browser';
import { NgModule } from '@angular/core';

import { AppComponent } from './app.component';

@NgModule({
  declarations: [
    AppComponent
  ],
  imports: [
    BrowserModule
  ],
  providers: [],
  bootstrap: [AppComponent]
})
export class AppModule { }
```

Structure du projet Angular

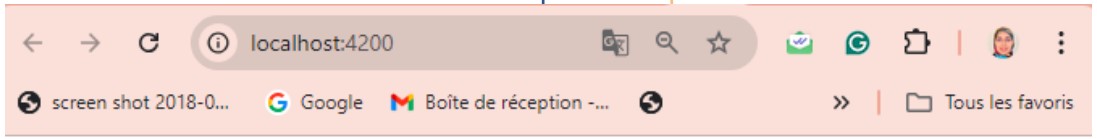


app.component.ts

```
import { Component } from '@angular/core';
@Component({
  selector: 'app-root',
  templateUrl: './app.component.html',
  styleUrls: ['./app.component.css']
})
export class AppComponent {
  title = 'app';
}
```

app.component.html

```
<div style="text-align:center">
  <h1>
    Welcome to {{title}}!!
  </h1>
</div>
```



Welcome to Polytechnique!!

Angular 2024

Les fichiers spec sont des tests unitaires pour vos fichiers source. La convention pour les applications Angular2 est d'avoir un fichier .spec.ts pour chaque fichier .ts. Ils sont exécutés à l'aide du framework de test javascript Jasmine via le programme de tâches Karma lorsque vous utilisez la commande '**ng test**'.

Type Script

- **TypeScript** est un langage de programmation libre et open source développée par **Microsoft** qui a pour but d'améliorer et de sécuriser la production de code JavaScript.
- C'est un sur-ensemble de **JavaScript** (c'est-à-dire que tout code JavaScript correct peut être utilisé avec TypeScript).
- Le code **TypeScript** est transcompilé en **JavaScript**, pouvant ainsi être interprété par n'importe quel navigateur web ou moteur JavaScript.
- Il a été cocréé par Anders Hejlsberg, principal inventeur de **C#**
- **TypeScript** permet un typage statique optionnel des variables et des fonctions, la création de classes et d'interfaces, l'import de modules, tout en conservant l'approche non-contraignante de JavaScript.
- Il supporte la spécification **ECMAScript 6.**

Type Script

Types des données dans TypeScript

- **Javascript est un langage non fortement typé mais dynamique**. Le type des données n'est identifié qu'à l'exécution du programme.
- TypeScript introduit la notion d'un typage un peu plus fort que pour le langage Javascript. Ce typage se manifeste par l'ajout de plusieurs types de bases ainsi qu'un typage statique.
- Rappelons qu'un langage est dit **de typage statique quand celui-ci vérifie les types des variables à la compilation**, alors qu'un langage dit de typage dynamique, effectue cette vérification à l'exécution.

Type Script

Syntaxe du typage

- Déclaration des variables

let variable: type;

- Exemple

  - let myVar: boolean;
  - let myVarinitialize: number = 2;
  - let bar = "hello world";
  - var mavariable= 123;

Type Script

Mot clé “let”

- var mavariable= 123;

```
if (true) {
   var mavariable= 456;
}
console.log(mavariable); //le résultat est: 456 et non pas 123
```

- mavariable a changé sa valeur dans tout le programme, car les accolades { } du if ne font pas la restriction des instructions.
- let mavariable= 123;

```
if (true) {
   let mavariable= 456;
}
console.log(mavariable); //le résultat est: 123 et non pas 456
```

- mavariable a changé sa valeur seulement à l'intérieur du bloc if(true) par la suite elle a repris sa valeur initiale (123) en dehors.

Type Script : Les types de bases (1)

TYPE DE DONNÉES	MOT-CLÉ	DESCRIPTION
Number	**number**	Il est utilisé pour représenter à la fois des nombres entiers et des nombres à virgule flottante.
Boolean	**boolean**	Représente true et false
String	**string**	Il est utilisé pour représenter une séquence de caractères
Void	**void**	Généralement utilisé sur les types de retour de fonction
Null	**null**	Il est utilisé lorsqu'un objet n'a aucune valeur
Undefined	**undefined**	Indique la valeur donnée à la variable non initialisée
Any	**Any**	Si la variable est déclarée avec **any**, n'importe quel type de valeur peut être affecté à cette variable

Type Script : Les types de bases (2)

Exemples:

- let a: null = null;
- let e: undefined = undefined;
- let f: boolean = true;
- let anInt: number = 6;
- let decimal: number = 2.9;
- let hex: number = 0xa5d2; *// Hexa-Decimal*
- let binary: number = 0b1010; *// Binaire*
- let octal: number = 0o762; *// Octal*
- let simple: string = 'string simple quote';
- let double: string = "string avec double quote";

Type Script : Les types de bases (3)

- **ARRAY:** Il existe deux manières équivalentes de définir les types de tableau: Array et T[]

```
let names: string[] = ["Pierre","Paul","Jaques"];
let otherNames: Array<string> = ["Pierre","Paul","Jaques"];
```

- **Objet:** Un objet est une instance qui contient un ensemble de paires clé-valeur. Les valeurs peuvent être des valeurs scalaires ou des fonctions ou même un tableau d'autres objets.

```
var object_name = { key1: "value1", //scalar value key2: "value", key3: function() { //functions }, key4:["content1", "content2"] //collection };
```

- **ENUM**

```
enum Couleur{Red = 0, Blue, Green }
let c: Couleur= Couleur.Red;
```

Par défaut, la première valeur de l'*enum* sera 0, toutefois il est possible de préciser la valeur de chaque élément de l'*enum*.

Type Script : Les Expressions Intégrées

- Les variables dans TypeScript peuvent êtres déclaré de deux façons (Explicite ou implicite).

➢ Explicite : il faut mentionner le type de la variable à déclarer

```
Var foo: number = 123;
```

➢ Implicite : la variable prend le type de sa valeur.

```
var foo = 123;

foo = 'Bonjour'; //Erreur
```

➔ Ici la variable foo est implicitement un number (entier)

Type Script : Les fonctions

- Une fonction est un ensemble d'instructions pour effectuer une tâche spécifique. Les fonctions organisent le programme en blocs de code logiques. Une fois définies, les fonctions peuvent être appelées pour accéder au code.
- Les fonctions facilitent la lecture et la maintenance du code du programme.
- Une déclaration de fonction indique au compilateur le nom, le type de retour et les paramètres d'une fonction. Une définition de fonction fournit le corps réel de la fonction.

```
function startRace(race: Race): Race {
race.status = RaceStatus.Started;
return race;
}
```

```
function startRace(race: Race): void {
race.status = RaceStatus.Started;
}
```

Type Script : Les classes

- On peut définir une classe en respectant la structure suivante:

```
export class Batiment {
  hauteur: number;
  largeur: number;
  constructor(x: number, y: number) {
     this.hauteur = x;
     this.largeur = y;
  }
  add(Batiment : b) {          // une méthode add(,)
     return new Batiment (this.x + b.x, this.y + b.y);
  }
}
var p1 = new Batiment (0, 10);
var p2 = new Batiment (10, 20);
var p3 = p1.add(p2); // Result est {x:10,y:30}
```

Type Script : Héritage

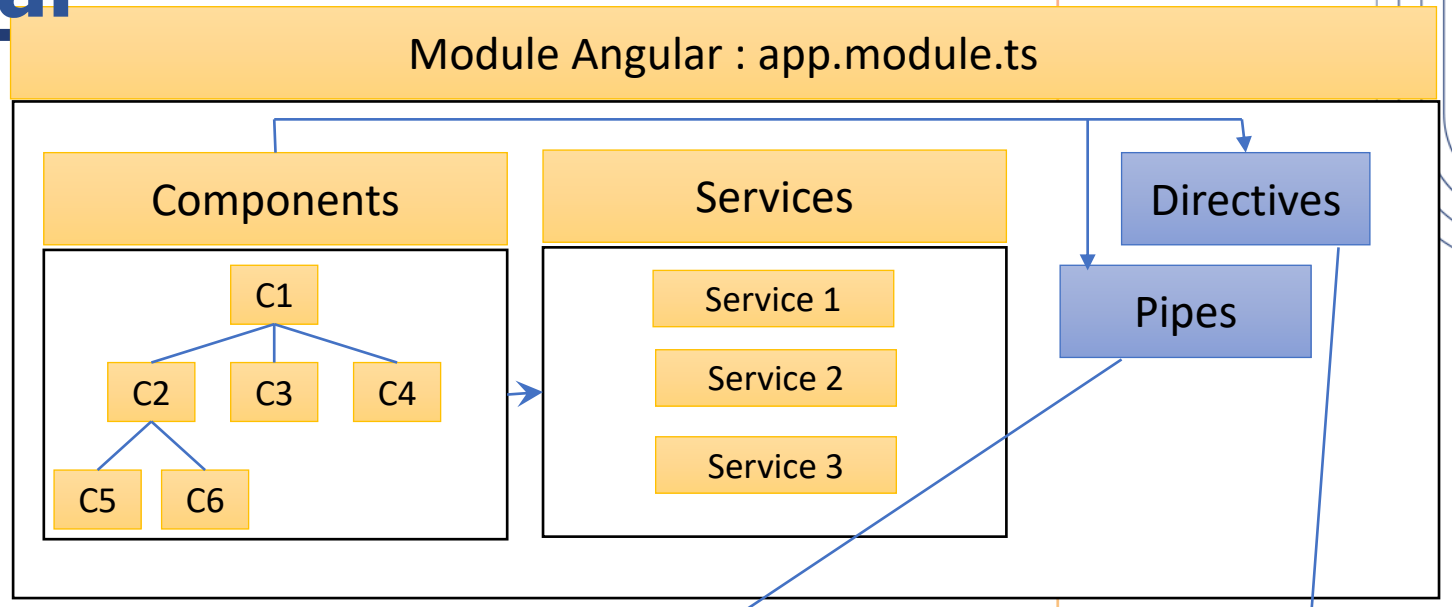
- Dans TypeScript comme dans C# ou Java il supporte l'héritage simple et non pas multiple. L'héritage se fait en utilisant le mot clé « extend »

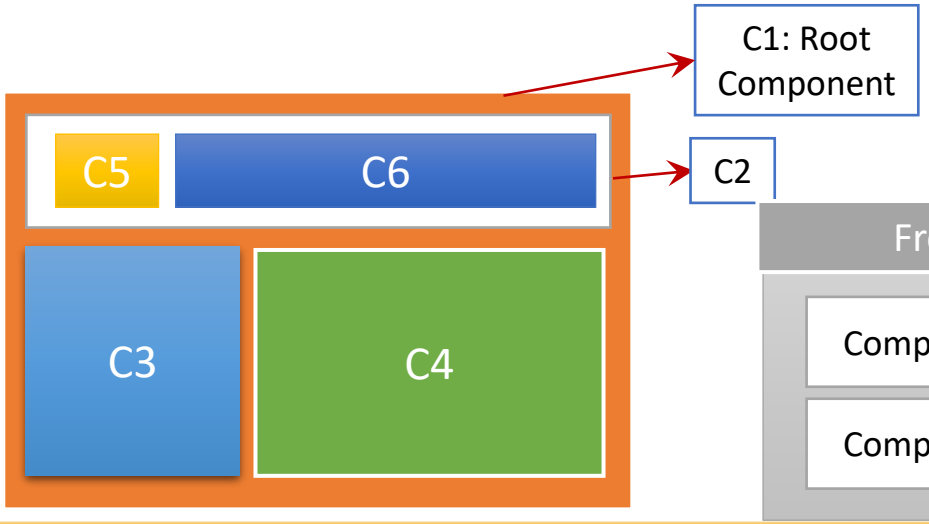
```
export class Maison extends Batiment {
    static nbTotalMaisons = 0
    nbChambres: number;
    constructor(x: number, y: number, z: number) {
        super(x, y);
        this.nbChambres = z;
    }
}
```

TypeScript nous donne la possibilité, aussi, de déclarer des variables globales en utilisant le mot clé static.

Architecture de Angular

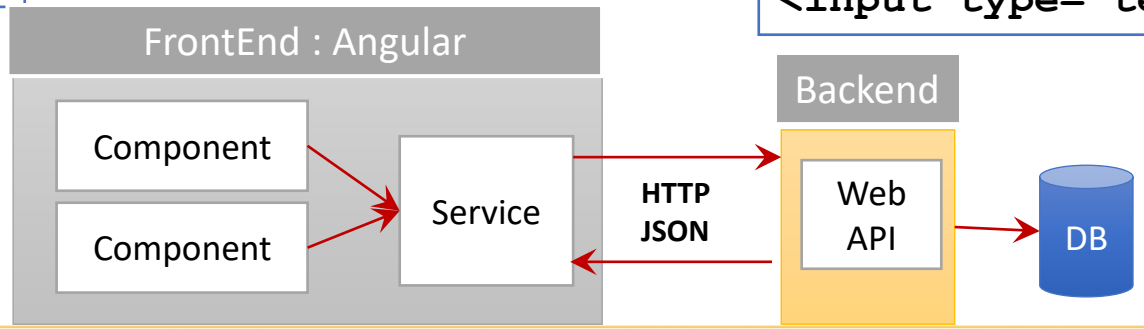
- Une application Angular se compose de :
  - Un à plusieurs modules dont un est principal.
  - Chaque module peut inclure :
    - Des composant web : La partie visible de la ‘application Web (IHM)
    - Des services pour la logique applicative. Les composants peuvent utiliser les services via le principe de l’injection des dépendances.
    - Les directives : un composant peut utiliser des directives
    - Les pipes : utilisés pour formater l’affichage des données dans les composants.






```
{{dateNaissance | date : 'dd/MM/yyyy'}}
```

```
<input type="text"  required />
```



Modules

- Les applications angulaires sont modulaires
- Angular possède son propre système de modularité appelé modules angulaires ou **NgModules**.
- Chaque application Angular possède au moins une classe de module angulaire: le module racine, appelé classiquement **AppModule**.
- Un module angulaire est une classe avec un décorateur @NgModule.
- Les décorateurs sont des fonctions qui modifient les classes JavaScript.
- Angular possède de nombreux décorateurs qui attachent des métadonnées aux classes pour configurer et donner le sens à ces classes.

src/app/app.module.ts

```
import { NgModule } from '@angular/core';
import { BrowserModule } from '@angular/platform-browser';
@NgModule({
                imports: [ BrowserModule ],
                providers: [ Logger ],
                declarations: [ AppComponent ],
                exports: [ AppComponent ],
                bootstrap: [ AppComponent ]
 })
export class AppModule { }
```

Modules : @NgModule

- **@NgModule** est un décorateur qui pend en parameter un objet javascript qui contient des métadonnes don't les propriétés décrivent le module. Les propriétés les plus importantes sont:
  - **declarations** : la classe représentant le module. Angular a trois types de classes de modules : components, directives, and pipes.
  - **exports** – Pour exporter les classes utilisables dans d'autres modules.
  - **imports** – Pour importer d'autres modules.
  - **providers** – Pour déclarer les fabriques de services.
  - **bootstrap** – Pour declarer le corposant Racine du module. Seul le module racine doit définir cette propriété.

src/app/app.module.ts

```
import { NgModule } from '@angular/core';
import { BrowserModule } from '@angular/platform-browser';
@NgModule({
                imports: [ BrowserModule ],
                providers: [ Logger ],
                declarations: [ AppComponent ],
                exports: [ AppComponent ],
                bootstrap: [ AppComponent ]
 })
export class AppModule { }
```

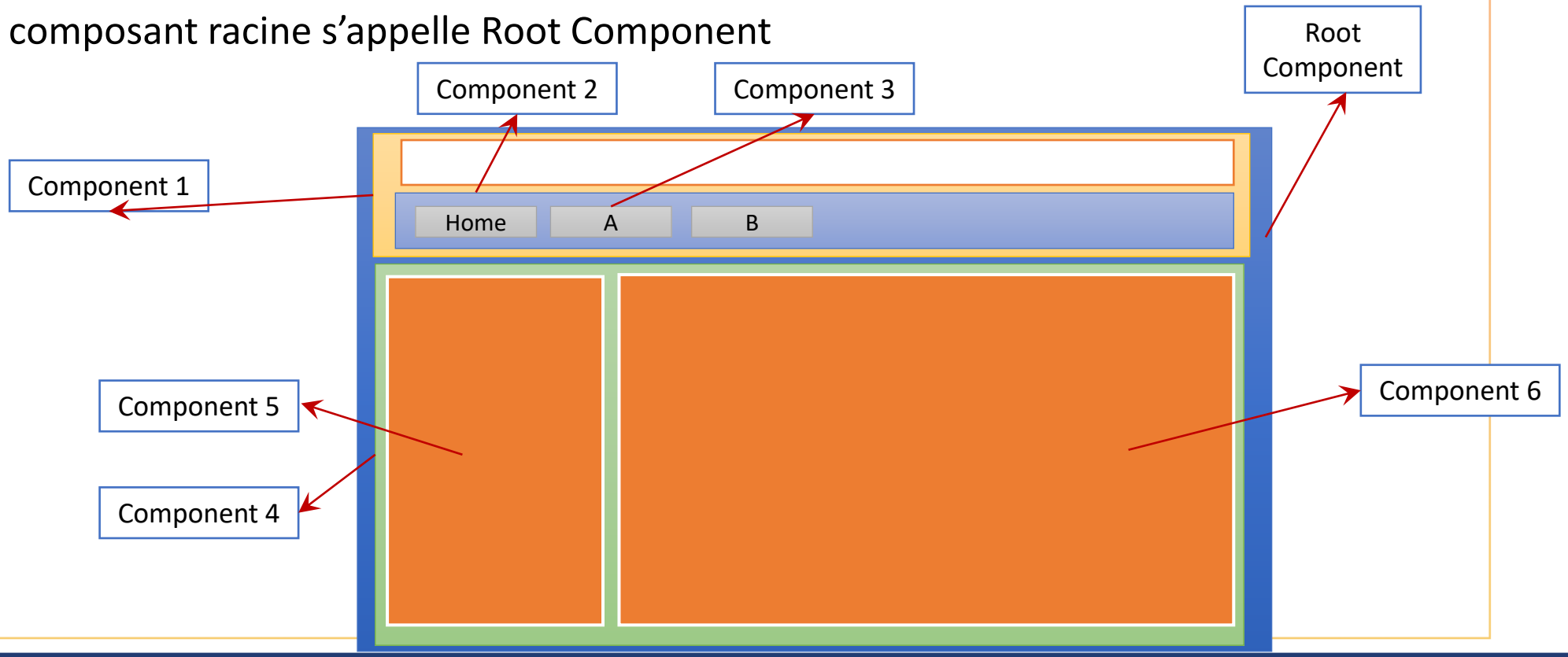
Démarrage de l'application

- Le module racine est démarré dans le fichier main.ts
- Par défaut le module racine s'appelle AppModule

```
import { enableProdMode } from '@angular/core';
import { platformBrowserDynamic } from '@angular/platform-browser-dynamic';

import { AppModule } from './app/app.module';
import { environment } from './environments/environment';

if (environment.production) {
  enableProdMode();
}

platformBrowserDynamic().bootstrapModule(AppModule);
```

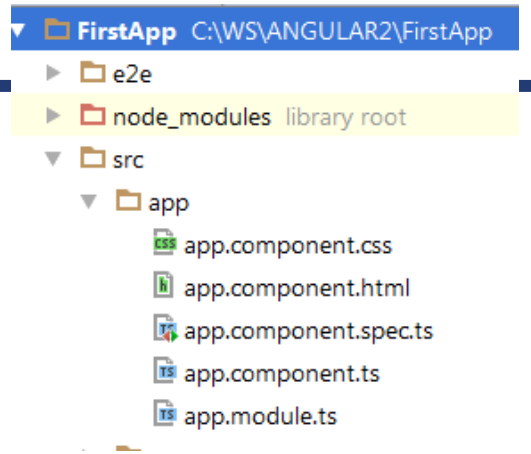
Components

- Les composants sont des éléments importants dans Angular.
- L'application est formée par un ensemble de composants.
- Chaque composant peut imbriquer d'autres composants définissant ainsi une structuré hiérarchique.
- Le composant racine s'appelle Root Component



Components

- Chaque composant se compose principalement des éléments suivants:
  - HTML Template : représentant sa vue
  - Une classe représentant sa logique métier
  - Une feuille de style CSS
  - Un fichier spec sont des tests unitaires Ils sont exécutés à l'aide du framework de test javascript Jasmine via le programme de tâches Karma lorsque vous utilisez la commande '**ng test**'.
- Les composants sont faciles à mettre à jour et à échanger entre les différentes parties des applications.



app.component.ts

```
import { Component } from '@angular/core';
@Component({
  selector: 'app-root',
  templateUrl: './app.component.html',
  styleUrls: ['./app.component.css']
})
export class AppComponent {
  title = 'app';
}
```

app.component.html

```
<div style="text-align:center">
  <h1>
    Welcome to {{title}}!!
  </h1>
</div>
```

app.component.css

Création de nouveaux composants

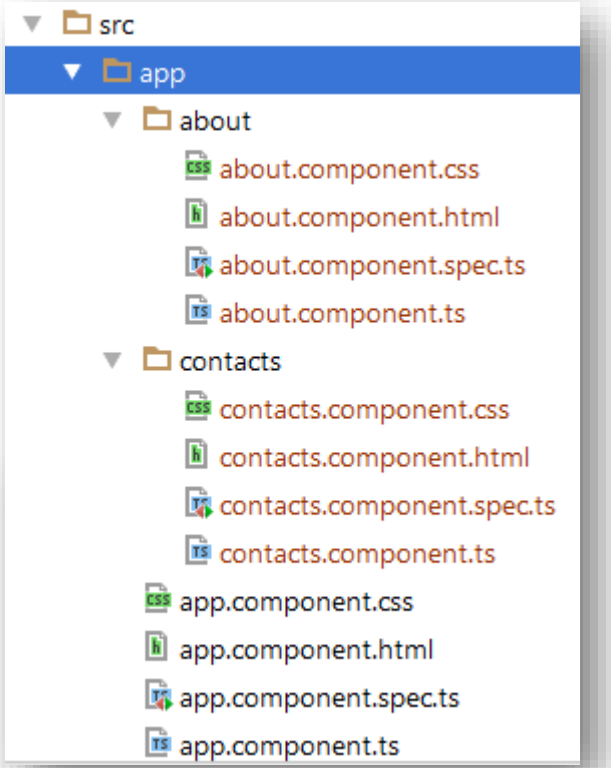
- Pour créer facilement des composants Angular, on peut utiliser à nouveau la commande ng comme suit :
  - **ng generate component NomComposant**
- Dans notre exemple, nous allons créer deux composants : about et contacts

Terminal

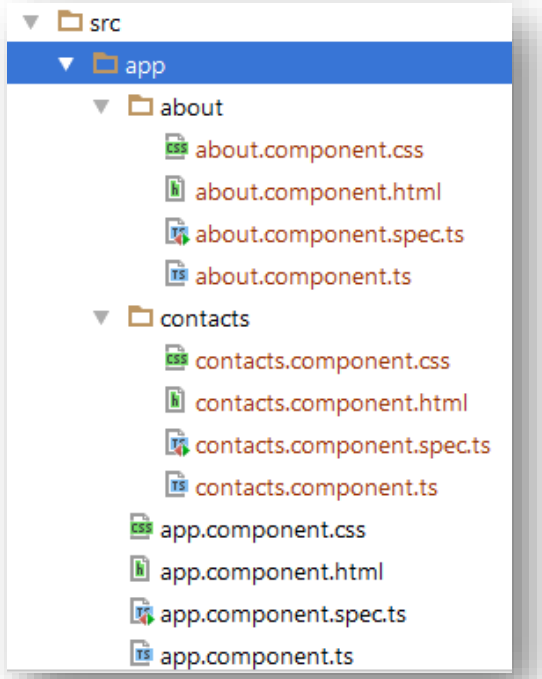
```
C:\WS\ANGULAR2\FirstApp>ng generate component about
installing component
  create src\app\about\about.component.css
  create src\app\about\about.component.html
  create src\app\about\about.component.spec.ts
  create src\app\about\about.component.ts
  update src\app\app.module.ts
```

Terminal

```
C:\WS\ANGULAR2\FirstApp>ng generate component contacts
installing component
  create src\app\contacts\contacts.component.css
  create src\app\contacts\contacts.component.html
  create src\app\contacts\contacts.component.spec.ts
  create src\app\contacts\contacts.component.ts
  update src\app\app.module.ts
```

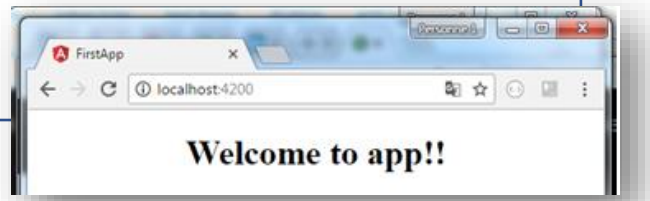


Structure du composant about



about.component.ts

```
import { Component, OnInit } from '@angular/core';
@Component({
  selector: 'app-about',
  templateUrl: './about.component.html',
  styleUrls: ['./about.component.css']
})
export class AboutComponent implements OnInit {
  constructor() { }
  ngOnInit() {
  }
}
```



about.component.html

```
<p>
  about works!
</p>
```

about.component.css

@Component

- Un composant est une classe qui possède le décorateur **@Component**
- Ce décorateur possède les propriétés suivantes :
  - **selector** : indique la déclaration qui permet d'insérer le composant dans le document HTML. Cette déclaration peut être :
    - Le nom de la balise associé à ce composant
      - selector : app-about
      - Dans ce cas le composant sera inséré par : <app-about></app-about>
    - Le nom de l'attribut associé à ce composant :
      - selector : [app-about]
      - Dans ce cas le composant sera inséré par : <div app-about>
    - Le nom de la classe associé à ce composant :
      - selector : .app-about
      - Dans ce cas le composant sera inséré par :
  - template ou templateUrl :
    - template : permet de définir dans à l'intérieur du décorateur le code HTML représentant la vue du composant
    - templateUrl : permet d'associer un fichier externe HTML contenant la structure de la vue du composant
  - styleUrls : spécifier les feuilles de styles CSS associées à ce composant

Déclaration du composant

- Pour utiliser un composant, ce dernier doit être déclaré dans le module :

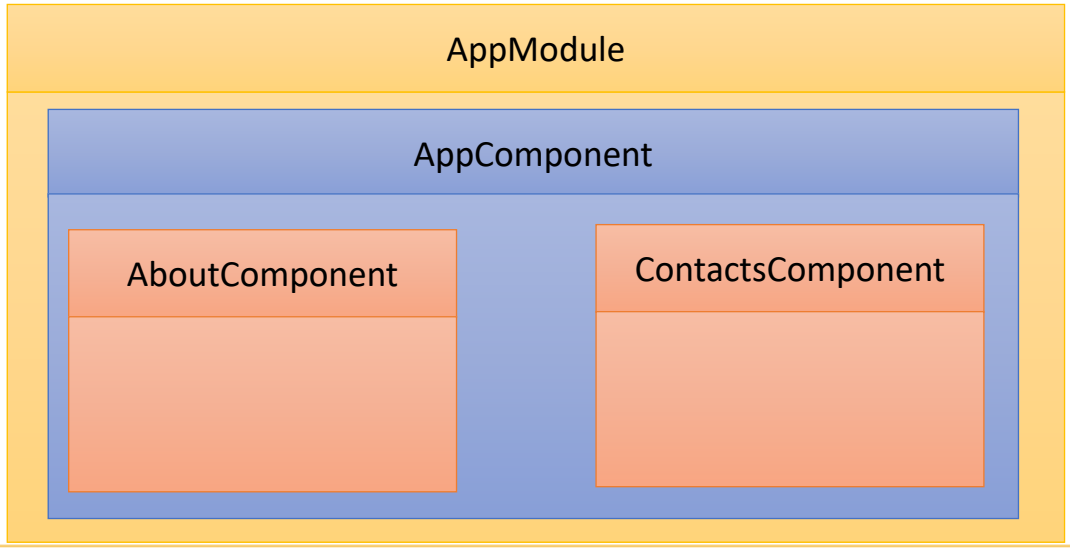
App.module.ts

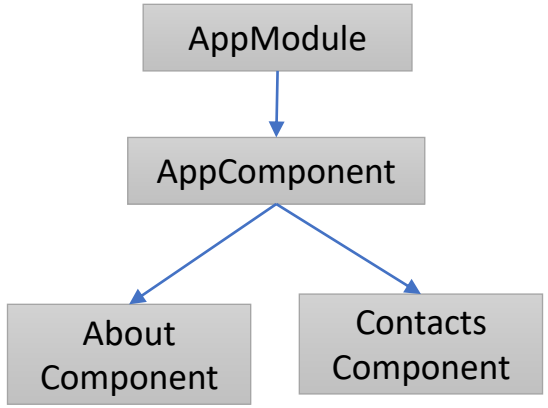
```
import { BrowserModule } from '@angular/platform-browser';
import { NgModule } from '@angular/core';

import { AppComponent } from './app.component';
import { AboutComponent } from './about/about component';
import { ContactsComponent } from './contacts/contacts.component';

@NgModule({
  declarations: [
     AppComponent,
     AboutComponent,
     ContactsComponent
   ],
  imports: [
    BrowserModule
  ],
  providers: [],
  bootstrap: [AppComponent]
})
export class AppModule { }
```

Utilisation composant

- Un composant peut être inséré dans n'importe que partie HTML de l'application en utilisant son selecteur associé.
- Dans cet exemple les deux composants générés sont insérés à l'intérieur du composant racine AppComponent

```
<div style="text-align:center">
  <h1>
    Welcome to {{title}}!!
  </h1>
  <app-about></app-about>
  <div app-contacts></div>
</div>
```





Important

- ✓ **npm –version** : afficher la version courante de gestionnaire de paquet
- ✓ **npm install –g @angular/cli** : installer angular, l'option 'g' pour dire indiquer que
- l'installation sera global
- ✓ **ng version** ou bien **ng v** : afficher la version courant d'Angular CLI
- ✓ **ng new <nom_application>** : créer une nouvelle application
- ✓ **npm install** : installer les paquets npm nécessaires.
- ✓ **npm serve** : pour lancer l'application Angular.
- ✓ **ng generate component NomComposant** : pour générer un nouveau composant